

Jour 2 – Branches, fusions et collaboration avancée

Table des matières générale des trois jours (rappel)

Jour	Contenu	Durée
Jour 1 – Introduction au versionning	Installation de Git, création d'un dépôt local, premiers commits, historique, dépôt distant, synchronisation.	7 h
Jour 2 – Branches, fusions et collaboration	Bases et stratégies de branches, gestion des branches locales/distant, fusions (fast-forward et 3-way), résolution de conflits, tags et versions, mini-projet collaboratif.	7 h
Jour 3 – Outils avancés et workflows d'équipe	Hooks, CI/CD, revues de code, sous-modules, bonnes pratiques, projet fil rouge final et ouverture.	7 h

Ce fichier détaille la **deuxième journée**, séquence par séquence. La progression respecte le référentiel RNCP39608BC03FM03 (C21 et C22), alterne explications courtes et travaux pratiques, favorise la participation et s'adapte au rythme des débutants comme des plus rapides.

Séquence 2.1 : Recap' et introduction aux branches (1 h)

Objectifs pédagogiques

- Consolider les acquis de la première journée et répondre aux dernières questions.
- Comprendre ce qu'est une branche dans Git et pourquoi elle est essentielle dans un système de gestion de versions décentralisé.
- Savoir créer, lister et changer de branche localement.

Explications du formateur (10 min max)

Qu'est-ce que c'est ? Une branche représente une ligne de développement indépendante. C'est un pointeur léger vers un commit spécifique : de nouveaux commits ajoutés dans une branche constituent un « fork » de l'historique ¹. Git encourage les workflows où l'on crée et fusionne des branches fréquemment : les branches sont légères et la bascule est quasi instantanée ².

À quoi ça sert ? Les branches permettent de développer des fonctionnalités ou de corriger des bugs sans affecter la version principale. Elles favorisent l'expérimentation et la collaboration : chaque développeur peut travailler sur sa propre branche puis intégrer ses changements dans la branche partagée quand ils sont prêts ¹. En projet, on trouve souvent des branches « feature », « bugfix » ou « hotfix ».

Comment on fait ?

1. **Créer une nouvelle branche** : `git branch nom_de_branche` crée un nouveau pointeur vers le commit actuel ³ . Cela ne change pas la branche active.
2. **Lister les branches** : `git branch` affiche toutes les branches locales ⁴ ; `git branch -a` liste également les branches distantes ⁵ .
3. **Changer de branche** : `git checkout nom_de_branche` déplace le pointeur spécial `HEAD` vers la branche choisie et modifie l'arborescence de travail ⁶ .

Exemple concret et terrain : Sur un projet e-commerce, on souhaite ajouter un module de paiement. On crée une branche `feature/paiement` pour travailler sans risquer de casser le site en production. Une fois la fonctionnalité testée, on fusionnera cette branche dans `main` . Cette approche s'applique à tout projet d'équipe (développeurs, chefs de projet, intégrateurs) et favorise la traçabilité.

Sujet de recherche/action (25 min)

Exercice : Création et exploration de branches

Suivez ce pas-à-pas en binôme ou en petit groupe. Prenez des notes ; vous utiliserez ces étapes pour le mini-projet.

1. **Installer Git et préparer le dépôt (si nécessaire)** : reprenez le dossier `exo_versioning` créé la veille ou créez un nouveau dossier avec `mkdir exo_branches && cd exo_branches` . Initialisez le dépôt : `git init` .
2. **Créer un fichier** `story.txt` avec un court contenu (un mini scénario). Ajoutez et validez : `git add story.txt` puis `git commit -m "Initialisation de l'histoire"` .
3. **Créer une branche** `nouveau-personnage` : `git branch nouveau-personnage` . Affichez la liste des branches et constatez que `HEAD` pointe toujours sur `master` .
4. **Basculez sur la branche** : `git checkout nouveau-personnage` . Modifiez `story.txt` en ajoutant une nouvelle phrase. Faites `git add .` puis `git commit -m "Ajout du personnage"` .
5. **Revenez sur** `master` : `git checkout master` . Ouvrez `story.txt` pour voir qu'il ne contient pas le personnage. Affichez l'historique succinct avec `git log --oneline --decorate` pour visualiser les deux branches ⁷ .
6. **Préparez une restitution** (oral + mini-document ou capture d'écran) expliquant l'impact de chaque commande et le rôle des branches.

Modalités de restitution

- **Orale en classe ou visioconférence** : chaque groupe présente en 3 min les étapes clés et ce qu'il a observé (différences de contenu entre les branches, position du `HEAD`).
- **Trace écrite** : un participant par groupe rédige un mini-compte-rendu (docs partagés sur Google Doc/Notion ou un schéma sur Miro) qui sera ajouté au livret de formation.

Correction et synthèse par le formateur (15 min)

- Vérifier que chaque équipe a créé une branche et qu'elle a bien basculé avec `git checkout` .
- Mettre en évidence l'importance d'un message de commit clair et descriptif ⁸ .
- Valoriser les initiatives (groupes qui ont testé d'autres options, par ex. `git branch -d` pour supprimer une branche) et recadrer les erreurs (ex. création de branche sans commit initial).

- **Synthèse** : Git permet de créer des branches à la volée, de tester des idées sans risque et de basculer rapidement d'un contexte à l'autre ² .

Ce qu'il faut retenir

- Une branche est un pointeur vers un commit et représente une ligne de développement indépendante ¹ .
- Git encourage la création et la fusion fréquentes de branches ; ces opérations sont rapides et légères ² .
- Utiliser `git branch`, `git checkout` et `git log --oneline --decorate` pour gérer et explorer les branches.

Conseils pour l'animateur

- Prévoir 10 min pour répondre aux questions en début de séquence et s'assurer que tout le monde a Git installé.
- Circuler entre les groupes pour aider à interpréter la sortie de `git log` et clarifier la notion de `HEAD` .
- Pour les apprenants en difficulté, proposer un support pas-à-pas imprimé ou partagé. Pour les plus rapides, suggérer d'expérimenter les options `git branch -m` (renommage) et `git branch -d` (suppression) ⁹ .
- En distanciel, utiliser un outil de partage d'écran et des breakout rooms pour le travail en binôme.

Séquence 2.2 : Stratégies et gestion des branches locales/distant (1 h 30)

Objectifs pédagogiques

- Comprendre les bonnes pratiques de nomination et d'organisation des branches (feature, bugfix, release).
- Appréhender la différence entre branches locales, branches distantes et branches de suivi.
- Apprendre à pousser (`push`), récupérer (`fetch` / `pull`) et supprimer des branches distantes ¹⁰ .

Explications du formateur (15 min)

Qu'est-ce que c'est ? Dans Git, une branche locale n'existe que sur votre machine. Une branche distante est hébergée sur un serveur (GitHub, GitLab...) et partagée avec l'équipe. Lorsqu'on clone un dépôt, Git crée des branches *de suivi* (`origin/branche`) qui pointent vers les branches distantes, mais on ne peut pas les modifier directement ¹¹ .

À quoi ça sert ? Utiliser des conventions de nommage (ex. `feature/...`, `fix/...`, `release/...`) facilite la collaboration et la revue de code. Les branches distantes permettent de partager son travail. Pousser uniquement les branches pertinentes évite d'exposer des développements expérimentaux ¹² .

Comment on fait ?

1. **Nommage** : adopter une convention simple (ex. `feature/login`, `bugfix/panier`) et l'expliquer à l'équipe.
2. **Pousser une branche** : `git push origin branche` transmet la branche locale vers le serveur ¹³.
3. **Récupérer les branches distantes** :
4. `git fetch` met à jour les pointeurs `origin/*` sans modifier votre travail ¹⁴.
5. `git pull` est équivalent à `git fetch` suivi d'un `git merge` sur la branche courante ¹⁵.
6. **Suivre une branche distante** : `git checkout --track origin/branche` crée une branche locale liée à la branche distante ¹⁶. Ainsi, `git pull` saura automatiquement sur quelle branche distante se synchroniser.
7. **Supprimer une branche distante** : `git push origin --delete branche` supprime la branche sur le serveur ¹⁷.

Exemple concret : Dans un projet de création d'un site vitrine, vous travaillez sur la fonctionnalité "Contact" dans la branche `feature/contact`. Une fois terminée et fusionnée dans `main`, vous supprimez la branche localement (`git branch -d feature/contact`) puis à distance (`git push origin --delete feature/contact`).

Exercices pratiques (45 min)

Exercice : Gérer des branches locales et distantes

1. **Préparation** : utilisez le dépôt `exo_branches` de la séquence précédente. Créez une branche `feature/fin` sur `master` et ajoutez-y une ligne finale dans `story.txt`. Commitez.
2. **Poussez la branche** sur GitHub : `git push origin feature/fin`. Si nécessaire, créez le dépôt distant depuis l'interface GitHub (bouton "New repository"), puis lier avec `git remote add origin <URL>` et `git push -u origin master` (si ce n'est pas déjà fait).
3. **Simulez un coéquipier** : ouvrez une nouvelle copie du dépôt (ou demandez à un binôme) et exécutez `git fetch origin` puis `git checkout --track origin/feature/fin`. Ajoutez une phrase et faites un commit.
4. **Poussez ces modifications** (`git push origin feature/fin`). Revenez sur votre dépôt initial et exécutez `git pull` pour récupérer le travail du coéquipier.
5. **Supprimez la branche** sur le serveur une fois la fonctionnalité fusionnée (nous verrons la fusion dans la séquence suivante) : `git push origin --delete feature/fin`.

Modalités de restitution

- **Démonstration écrite ou vidéo** : chaque groupe partage un mini tutoriel (capture d'écran ou vidéo Loom) montrant la création de la branche, le `push` et le `pull`.
- **Réponse orale** : expliquer la différence entre `git fetch` et `git pull` et quand utiliser chacune des commandes.

Correction et synthèse (30 min)

- Vérifier que les branches sont poussées et suivies correctement. Souligner que les branches locales ne sont pas synchronisées automatiquement et qu'il faut choisir lesquelles publier ¹².
- Valider la compréhension du concept de branche de suivi (`origin/branche`) et des conventions de nommage.

- Rappeler l'importance de supprimer les branches devenues inutiles afin d'éviter l'encombrement sur le serveur. ¹⁷ .

Ce qu'il faut retenir

- Les branches locales sont privées ; pour partager, il faut les pousser vers un dépôt distant avec `git push origin branche` ¹³ .
- `git fetch` récupère les références distantes sans modifier la branche courante ; `git pull` effectue également un merge ¹⁸ .
- Une branche de suivi est une branche locale associée à une branche distante, créée avec `git checkout --track origin/branche` ¹⁶ .
- Supprimer les branches terminées garde le projet propre à long terme.

Conseils pour l'animateur

- Mettre en place un dépôt GitHub commun pour la session afin que les apprenants puissent collaborer. Proposer un modèle de nommage des branches.
- Pour les moins à l'aise : fournir un pas-à-pas détaillé avec copies d'écran. Pour les plus avancés : leur demander d'expérimenter la création d'une branche locale avec un nom différent de la branche distante (`git push origin local:distante`) ¹⁹ .
- Utiliser un tableau partagé (Miro) pour cartographier les différentes branches et leurs statuts (en développement, en revue, fusionnée, supprimée).
- En remote, vérifier régulièrement la progression via un outil de partage de code (ex. GitHub Classroom, Replit).

Séquence 2.3 : Merging et stratégies de fusion (1 h 30)

Objectifs pédagogiques

- Comprendre comment fusionner des branches et la différence entre merge fast-forward et merge 3-way.
- Savoir préparer une fusion et choisir la bonne stratégie (`--no-ff`, squash...).
- Expérimenter la fusion de branches dans différents contextes (petite fonctionnalité vs longue fonctionnalité).

Explications du formateur (15 min)

Qu'est-ce que c'est ? La fusion (merge) est la méthode de Git pour rassembler des lignes de développement qui ont divergé. La commande `git merge` permet de prendre les branches indépendantes créées par `git branch` et de les intégrer dans une seule branche ²⁰ .

À quoi ça sert ? Quand plusieurs développeurs ou plusieurs branches travaillent sur des fonctionnalités différentes, il faut réunifier les changements pour construire une version cohérente du projet. La fusion combine les historiques de manière contrôlée et laisse une trace de l'intégration. Une bonne gestion des fusions facilite la relecture, l'intégration continue et la traçabilité.

Comment on fait ?

1. **Se positionner sur la branche réceptrice** (souvent `main`) : `git checkout main`. Toujours vérifier l'état avec `git status` ²¹ .

2. **Mettre à jour** la branche réceptrice : `git fetch` puis `git pull` pour obtenir les dernières modifications distantes ²² .
3. **Lancer la fusion** : `git merge <branche>` crée un commit de fusion qui combine l'historique des deux branches ²³ .
4. **Fast-forward vs 3-way merge** :
5. *Fast-forward* : lorsqu'il existe un chemin linéaire entre les branches, Git avance simplement le pointeur de la branche réceptrice vers le dernier commit de la branche fusionnée ²⁴ .
6. *3-way merge* : lorsque les branches ont divergé, Git crée un commit de fusion avec deux parents ²⁵ . Cette méthode est également utilisée lorsqu'on force la création d'un commit (option `--no-ff`).
7. **Option** `--no-ff` : `git merge --no-ff <branche>` génère toujours un commit de fusion, même si un fast-forward est possible, afin de conserver l'historique distinct de la branche ²⁶ .

Exemple concret : Dans une équipe, la branche `feature/contact` a une durée de vie courte : deux commits. La fusion vers `main` se fera en fast-forward car `main` n'a pas bougé depuis la création de `feature/contact`. À l'inverse, la branche `feature/abonnement` est restée active pendant plusieurs jours pendant que d'autres commits étaient réalisés sur `main`. Lorsqu'on la fusionne, Git crée un commit de fusion (3-way). Si l'équipe souhaite garder une trace visible de toutes les intégrations, elle pourra forcer un commit de fusion même pour des branches courtes avec `--no-ff`.

Exercices pratiques (45 min)

Exercice : Fusionner des branches (fast-forward et 3-way)

1. **Préparation** : Dans le dépôt `exo_branches`, assurez-vous que `main` est à jour (`git checkout main && git pull`).
2. **Création d'une branche courte** : `git checkout -b feature/short`. Modifiez `story.txt` (ajoutez une petite phrase) et faites deux commits. Revenez sur `main` et fusionnez : `git merge feature/short`. Observez que Git effectue un fast-forward (pas de commit de fusion). Vérifiez l'historique avec `git log --oneline --graph --all`.
3. **Création d'une branche longue** : `git checkout -b feature/long`. Ajoutez un fichier `chapitre2.txt` et faites un commit. Revenez sur `main`, modifiez `story.txt` (ajoutez une ligne) et validez. Revenez sur `feature/long`, modifiez `chapitre2.txt` puis validez. Revenez sur `main` et lancez `git merge feature/long`. Git crée un commit de fusion. Visualisez l'historique avec `git log --oneline --graph --all`.
4. **Forcer un commit de fusion** : Recommencez la fusion d'une petite branche en utilisant `--no-ff` pour générer un commit de fusion même si un fast-forward est possible.

Quiz / mini challenge (20 min)

- Petit quiz interactif via Kahoot ou Google Forms (5 questions) : reconnaître le type de merge selon un graphe d'historique, définir `--no-ff`, identifier la commande pour fusionner.
- Pour les plus rapides : créer un alias Git (`git config --global alias.lg "log --oneline --graph --all"`) pour visualiser les arbres rapidement.

Modalités de restitution

- Démonstration par un volontaire de la fusion fast-forward et 3-way sur vidéoprojecteur ou partage d'écran.

- Capture d'écran des graphes d'historique annotés sur un outil collaboratif (Miro ou Excalidraw) afin de garder une trace visuelle des types de fusion.

Correction et synthèse (10 min)

- Vérifier que l'historique est compris et que les notions de fast-forward et de merge commit sont assimilées.
- Souligner que les merges peuvent être préparés avec `git fetch` et que l'option `--no-ff` laisse un historique plus lisible dans certains workflows (ex. GitFlow).

Ce qu'il faut retenir

- `git merge` intègre des lignes de développement indépendantes dans une branche ²⁰.
- Un merge fast-forward déplace simplement le pointeur de la branche réceptrice ²⁴ ; un merge 3-way crée un commit de fusion lorsqu'il y a divergence ²⁵.
- L'option `--no-ff` force la création d'un commit de fusion, utile pour garder trace des branches même lorsqu'un fast-forward est possible ²⁶.

Conseils pour l'animateur

- Préparer un dépôt de démonstration pour illustrer les deux types de fusion. Utiliser des schémas (ex. `git log --graph`) pour aider à la compréhension visuelle.
- Pour les apprenants en difficulté, proposer un pas-à-pas supplémentaire ou un tutoriel vidéo. Pour les plus avancés, proposer de tester des stratégies comme `rebase` (explication succincte sans aller trop loin), ou de combiner plusieurs merges.
- Rester attentif à la gestion du temps ; la pratique et l'observation des historiques prennent du temps.

Séquence 2.4 : Résolution de conflits (1 h)

Objectifs pédagogiques

- Identifier et comprendre les situations de conflits lors d'une fusion.
- Savoir utiliser les outils de Git pour détecter, analyser et résoudre les conflits.
- Développer une méthode pour communiquer et documenter la résolution.

Explications du formateur (10 min)

Qu'est-ce qu'un conflit ? Lors d'une fusion, si Git ne peut pas combiner automatiquement des modifications (par exemple, deux branches modifient la même ligne), il signale un conflit. L'utilisateur doit choisir quelles modifications conserver. Les fichiers conflictuels contiennent des marqueurs `<<<<<<<`, `=====` et `>>>>>>>` indiquant les sections concernées ²⁷.

À quoi ça sert ? Résoudre un conflit garantit que le code final correspond à l'intention de l'équipe. La maîtrise de ce processus est indispensable pour travailler à plusieurs et maintenir un historique propre.

Comment on fait ?

1. **Détecter les conflits** : après un `git merge`, Git indique les fichiers en conflit. Utiliser `git status` pour lister les fichiers non fusionnés ²⁷.

2. **Ouvrir le fichier en conflit** et repérer les marqueurs `<<<<<<`, `=====`, `>>>>>>`. Les modifications provenant de la branche courante se trouvent entre `<<<<<<` et `=====`, celles de la branche fusionnée après `=====` ²⁷.
3. **Choisir les modifications** à garder ou combiner les deux. Supprimer les marqueurs après avoir effectué le choix. Sauvegarder le fichier.
4. **Valider la résolution** : `git add <fichier>` marque le conflit comme résolu, puis `git commit -m "Résolution de conflit sur <fichier>"` enregistre la résolution ²⁷.

Exemple concret : Dans `feature/long`, deux développeurs modifient la même phrase de `chapitre2.txt`. Lors de la fusion, Git signale un conflit. Le groupe discute et décide de combiner les deux phrases. Ils éditent le fichier, retirent les marqueurs, ajoutent la version finale, puis enregistrent la résolution.

Exercices pratiques (35 min)

Exercice : Provoquer et résoudre un conflit

1. **Préparation** : sur votre dépôt `exo_branches`, créez deux branches à partir de `main` : `conflict1` et `conflict2` (`git checkout -b conflict1`, puis `git checkout main` et `git checkout -b conflict2`).
2. **Dans** `conflict1`, modifiez la première phrase de `story.txt` et committez (`git add . && git commit -m "Modification dans conflict1"`).
3. **Dans** `conflict2`, modifiez la même phrase de `story.txt` mais différemment et committez.
4. Revenez sur `main`, faites un `git merge conflict1` pour intégrer la première branche. Puis lancez `git merge conflict2` et observez l'apparition du conflit.
5. Utilisez `git status` pour repérer les fichiers en conflit. Ouvrez le fichier et résolvez le conflit. Une fois satisfait, exécutez `git add story.txt` puis `git commit -m "Résolution conflit story.txt"`.
6. Vérifiez l'historique (`git log --oneline --graph --all`) pour voir le commit de résolution.

Modalités de restitution

- **Démonstration en direct** : un volontaire partage son écran et montre la résolution des conflits.
- **Fiche technique** : chaque groupe rédige un tutoriel pas-à-pas (ou enregistre un screencast) pour expliquer la méthode de résolution et les bonnes pratiques.

Correction et synthèse (15 min)

- Valider que les marqueurs ont été supprimés et que le fichier compile/fonctionne après résolution.
- Insister sur la nécessité de communiquer et de documenter les décisions de résolution dans le message de commit (ex. "Résolution de conflit : conservation des deux versions combinées").
- Souligner que Git n'impose pas de choisir l'une ou l'autre version ; on peut combiner les deux ou créer une solution différente.

Ce qu'il faut retenir

- Un conflit survient lorsque Git ne peut pas fusionner automatiquement des changements contradictoires ²⁷.
- Utiliser `git status` pour repérer les fichiers concernés et éditer les marqueurs `<<<<<<`, `=====`, `>>>>>>` pour résoudre le conflit ²⁷.

- Après résolution, ajouter (`git add`) puis committer les fichiers modifiés pour finaliser la fusion.

Conseils pour l'animateur

- Préparer à l'avance un exemple de conflit pour montrer la démarche rapidement.
- Encourager l'entraide : organiser les équipes en binômes, un participant guidant l'autre.
- Pour les moins à l'aise, proposer de travailler dans un éditeur qui met en évidence les conflits (VS Code, GitKraken). Pour les plus avancés, suggérer d'explorer des outils de merge graphique (Meld, Kdiff3).
- Rappeler que la résolution de conflits est souvent une discussion de groupe : inciter à communiquer avant de trancher.

Séquence 2.5 : Tags et gestion des versions (1 h)

Objectifs pédagogiques

- Découvrir l'utilisation des tags pour marquer des versions stables ou des releases dans l'historique Git.
- Faire la différence entre tags légers et tags annotés, et comprendre quand les utiliser.
- Apprendre à créer, lister et partager des tags avec l'équipe.

Explications du formateur (10 min)

Qu'est-ce qu'un tag ? Un tag marque un point spécifique de l'historique, souvent utilisé pour signaler une version stable (v1.0, v2.0). Git permet deux types de tags : **léger** (simple pointeur) et **annoté** (stocke le nom du tagueur, la date et un message) ²⁸. Les tags annotés sont recommandés pour publier des releases officielles.

À quoi ça sert ? Les tags offrent une référence immuable pour revenir à une version connue du projet. Ils sont essentiels pour le déploiement, l'intégration continue, la génération de changelogs et la reproductibilité. Dans un métier de développeur ou de chef de projet, on se base sur les tags pour suivre l'évolution d'un produit.

Comment on fait ?

1. **Lister les tags :** `git tag` affiche tous les tags, `git show <tag>` montre les détails d'un tag (commit associé, message) ²⁹.
2. **Créer un tag léger :** `git tag v1.0` crée un pointeur vers le commit courant. ³⁰
3. **Créer un tag annoté :** `git tag -a v1.1 -m "Version 1.1"` enregistre également le nom de l'auteur et un message ²⁹.
4. **Pousser les tags :** `git push origin v1.1` pour partager un tag individuel, ou `git push origin --tags` pour pousser tous les tags.
5. **Supprimer un tag :** localement avec `git tag -d v1.1`, à distance avec `git push origin --delete v1.1`.

Exemple concret : Après avoir fusionné et validé la version 2.0 d'une application, vous créez un tag annoté `v2.0` avec un message qui décrit les principales fonctionnalités. Ce tag est poussé sur GitHub et utilisé par l'équipe DevOps pour déclencher le déploiement automatique.

Exercices pratiques (40 min)

Exercice : Créer et utiliser des tags

1. **Identification d'une version stable** : choisissez un commit stable dans votre dépôt `exo_branches` (par exemple, après la fusion de toutes les branches de la journée). Exécutez `git tag` pour vérifier qu'aucun tag n'existe.
2. **Création d'un tag léger** : `git tag v1.0`. Vérifiez qu'il apparaît avec `git tag` et visualisez le commit correspondant avec `git show v1.0`.
3. **Création d'un tag annoté** : `git tag -a v1.1 -m "Ajout de la fonctionnalité X"`. Affichez les informations avec `git show v1.1` (vous devriez voir votre nom, email, date et message) ²⁸.
4. **Pousser les tags** sur GitHub : `git push origin v1.1` puis `git push origin --tags`. Vérifiez sur l'interface GitHub que les tags sont visibles.
5. **Documentation** : rédigez une petite note décrivant la différence entre les versions taguées `v1.0` et `v1.1`, et comment revenir à une version taguée (ex. `git checkout v1.0`).

Modalités de restitution

- **Production écrite** : chaque groupe produit un court document (Google Doc) expliquant le processus de création et d'utilisation des tags, avec captures d'écran des commandes et du résultat.
- **Oral** : un membre par groupe présente l'importance des tags pour la mise en production et les releases.

Correction et synthèse (10 min)

- Vérifier que les tags ont bien été créés et qu'ils apparaissent sur GitHub.
- Souligner l'intérêt des tags annotés pour documenter une version et rappeler de toujours rédiger un message clair.
- Rappeler que les tags ne bougent jamais : si des corrections sont apportées après un tag, il faut créer une nouvelle version (ex. `v1.1`, `v1.2`).

Ce qu'il faut retenir

- Les tags marquent des versions stables du projet ; ils peuvent être légers ou annotés ²⁸.
- Créer un tag avec `git tag -a vX.Y -m "message"` et le pousser vers le dépôt distant. Utiliser `git show` pour consulter les détails ²⁹.
- Les tags sont immuables ; pour une nouvelle version, créer un nouveau tag.

Conseils pour l'animateur

- Illustrer les tags par des cas concrets (releases de versions, déploiements). Préparer un exemple de pipeline CI/CD qui se déclenche sur un tag (sans rentrer dans la technique avant le Jour 3).
- Pour les apprenants en difficulté, proposer un gabarit de message pour les tags annotés. Pour les plus avancés, introduire brièvement la signature des tags (`git tag -s`).
- Mettre en parallèle les tags et les versions officielles de logiciels connus (WordPress, Prestashop) afin d'ancrer les notions dans le réel.

Séquence 2.6 : Mini-projet collaboratif et rétrospective (1 h)

Objectifs pédagogiques

- Mettre en pratique l'ensemble des connaissances acquises sur les branches, fusions, résolutions de conflits et tags dans un contexte collaboratif.
- Favoriser l'entraide et la co-évaluation entre apprenants.
- Dresser un bilan de la journée et recueillir les retours pour ajuster la suite de la formation.

Déroulement (45 min)

1. **Constitution des groupes** : par équipes de 3 à 4 personnes. Chaque équipe choisit un thème (ex. rédaction d'un mini tutoriel, création d'une histoire interactive, etc.).
2. **Organisation du travail** :
3. Créez un dépôt GitHub partagé (par exemple `mini-projet-j2`). Chaque participant clone le dépôt et crée sa branche `feature/<son_nom>`.
4. Chaque membre ajoute sa contribution (texte dans un fichier, image, petit script...) et committe régulièrement.
5. Des fusions régulières doivent être effectuées. L'équipe s'assure que la branche `main` reste fonctionnelle et qu'aucune modification n'est perdue.
6. Si des conflits apparaissent, ils doivent être résolus collectivement et documentés.
7. À la fin, un tag annoté `v1.0` marque la version livrable du mini-projet.
8. **Documentation et restitution** :
9. Chaque groupe prépare une courte présentation (5 min) expliquant la stratégie de branches utilisée, comment les fusions ont été gérées, les conflits rencontrés, les tags créés et les bonnes pratiques retenues.
10. Les livrables (présentation, dépôt Git avec historique clair, tutoriels) sont ajoutés au livret de la formation.

Rétrospective (15 min)

- **Tour de table** : chaque apprenant partage un élément appris, une difficulté rencontrée et une suggestion pour améliorer les prochaines séquences.
- **Auto-évaluation et co-évaluation** : utilisation d'un tableau collaboratif (Miro ou Google Sheet) avec des critères simples (compréhension des branches, maîtrise du merge, gestion des conflits, création de tags). Chaque participant s'auto-évalue et évalue ses coéquipiers de manière bienveillante.

Ce qu'il faut retenir (points de synthèse de la journée)

- Les branches offrent un terrain de jeu sécurisé pour développer des fonctionnalités et Git encourage leur usage intensif ².
- La gestion des branches distantes et des branches de suivi est essentielle pour collaborer et partager son travail ¹⁰.
- Les fusions (fast-forward et 3-way) permettent d'intégrer les différents travaux et doivent être préparées avec soin ²⁰.
- Savoir résoudre les conflits calmement en analysant les marqueurs et en discutant avec l'équipe est une compétence clé ²⁷.
- Les tags permettent de jalonner l'histoire du projet et de marquer des versions stables ²⁸.
- La collaboration, la communication et l'entraide sont au cœur de la gestion de versions décentralisée.

Conseils pour l'animateur

- Laisser les équipes s'auto-organiser tout en s'assurant que chacun participe. Utiliser des rôles (scrum master, développeur, rapporteur) pour répartir les responsabilités.
- Encourager la co-évaluation et la reformulation pour renforcer la compréhension collective.
- Prévoir un temps tampon pour aider les groupes ayant rencontré un blocage, et proposer des mini-challenges bonus pour ceux qui avancent rapidement (ex. installer un outil graphique de merge, comparer `rebase` et `merge` de manière simplifiée).
- En distanciel, utiliser un outil de chat vocal (Discord) et des salles virtuelles pour les groupes. S'assurer que les supports (dépôts, docs, slides) sont accessibles à tous.

Pour aller plus loin

En clôture de cette deuxième journée, proposez des pistes de perfectionnement pour ceux qui souhaitent approfondir :

- Explorer différents workflows Git comme *GitFlow*, *Forking Workflow* ou *Feature Branch Workflow* ³¹.
- Se familiariser avec les revues de code via des Pull Requests (abordées en Jour 3).
- Expérimenter la commande `rebase` pour réécrire l'historique et comparer avec `merge`.
- Contribuer à des projets open source simples pour mettre en pratique la collaboration décentralisée.

¹ ³ ⁴ ⁵ ⁹ ¹⁷ ³¹ Git Branch | Atlassian Git Tutorial

<https://www.atlassian.com/git/tutorials/using-branches>

² ⁶ ⁷ Git - Branches in a Nutshell

<https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>

⁸ What is Version Control? · GitHub

<https://github.com/resources/articles/software-development/what-is-version-control>

¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁸ ¹⁹ Git - Remote Branches

<https://git-scm.com/book/en/v2/Git-Branching-Remote-Branches>

²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ²⁶ Git Merge | Atlassian Git Tutorial

<https://www.atlassian.com/git/tutorials/using-branches/git-merge>

²⁷ Resolving a merge conflict using the command line - GitHub Docs

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-using-the-command-line>

²⁸ ²⁹ ³⁰ Git - Tagging

<https://git-scm.com/book/en/v2/Git-Basics-Tagging>